

HACKATHON STUDY GUIDE · PROBLEM STATEMENT 11

Problem Statement 11 — M5 Retail Demand Forecasting Dataset Explained

Complete Dataset Guide: Files, Columns, Relationships, Forecasting Role, and Data Availability

3,049

PRODUCTS

10

STORES

30,490

TIME SERIES

1,969

CALENDAR DAYS

28

FORECAST HORIZON (DAYS)

Source of facts: DATASET_SUMMARY.md and the actual dataset investigation of [DATASET_m5-forecasting-accuracy/](#)

Scope: Dataset understanding only — no model built, no algorithm chosen, no feature engineering performed.

Audience: Someone comfortable with basic Python/ML, new to the M5 dataset.

Table of Contents

1. Executive Summary
2. Big-Picture Dataset Architecture
3. Dataset File #1 — sales_train_validation.csv
4. Dataset File #2 — sales_train_evaluation.csv
5. Dataset File #3 — calendar.csv
6. Dataset File #4 — sell_prices.csv
7. Dataset File #5 — sample_submission.csv
8. How the Five Files Work Together
9. Understanding the 30,490 Time Series
10. Date and Forecasting Timeline
11. The Future-Information Question
12. Data Characteristics We Discovered
13. Zero Sales / Intermittent Demand
14. Price Data Limitation
15. What Information Is Available at Forecast Time?
16. What We Are Not Given
17. Dataset → Forecasting Problem
18. Glossary
19. Final "Know This Before Modeling" Page

1. Executive Summary

What is this dataset? It is retail sales data from a real chain of stores. It records, for **3,049 products** sold across **10 stores** in **3 US states**, how many units of each product were sold **every single day** for over five years. Alongside the sales, we're given a calendar (with holidays and a food-assistance-benefit flag) and the weekly selling price of every product.

What are we trying to predict? For every one of the **30,490 product-store combinations**, how many units will sell on each of the **next 28 days**.

WHAT INFORMATION DO WE HAVE

- 1,941 days of actual historical daily sales for every product-store combination
- A calendar covering 1,969 days total — including the 28 days right after the sales history ends
- Weekly prices for every product-store combination — also covering those same 28 future days
- No promotion/discount data of any kind (confirmed absent — see Section 16)

What is the forecasting horizon? 28 days — from the day after the last known sale to the last day of the calendar file.

The core numbers, all verified from the actual files

Quantity	Value
Products	3,049
Stores	10
States	3 (California, Texas, Wisconsin)
Store-item time series	30,490 (= 3,049 × 10)
Calendar days covered	1,969 (2011-01-29 → 2016-06-19)
Historical sales through	2016-05-22
Forecast window	2016-05-23 → 2016-06-19
Forecast horizon	28 days

IN PLAIN LANGUAGE

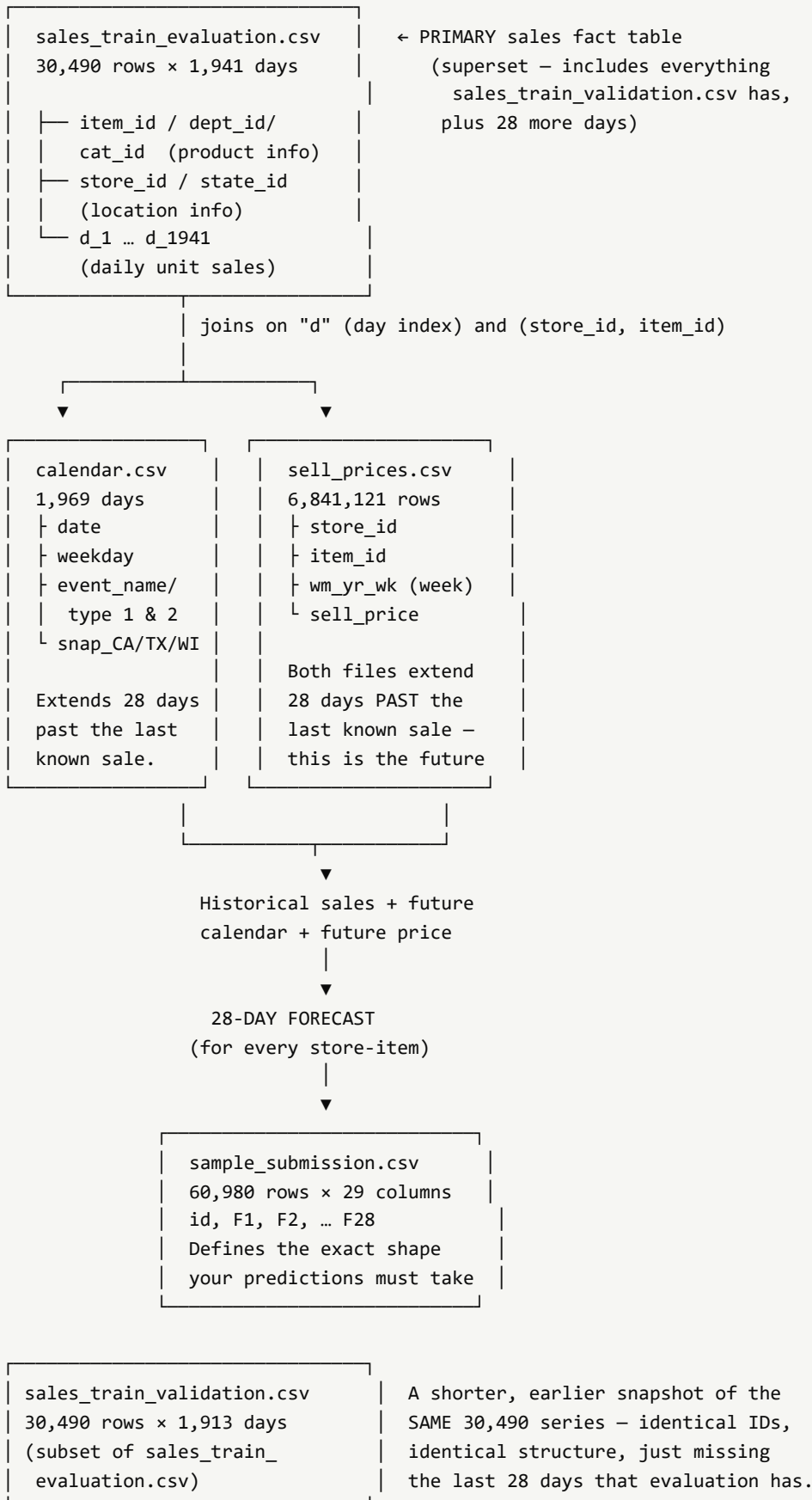
This is not "one product, one time series" forecasting. It's 30,490 separate daily time series that all need forecasting at once, most of which sell only occasionally (more in Section 13), with a calendar and price file that conveniently already tell us what the world looks like during the 28 days we need to predict — we just don't know the sales themselves yet.

2. Big-Picture Dataset Architecture

IN WORDS

`sales_train_evaluation.csv` is the main historical-sales table — one row per product-store combination, one column per day. `calendar.csv` and `sell_prices.csv` both attach extra context to those daily columns (calendar via the day index, price via the week index), and — importantly — both of them keep going for **28 days after the sales data stops**. That 28-day extension is exactly the forecast window, and it's why we already have calendar and price information for it. `sample_submission.csv` just tells us the exact output format expected. `sales_train_validation.csv` is not a separate dataset with new information — it's confirmed to be a strict subset (same IDs, same values, just 28 fewer days) of `sales_train_evaluation.csv`.

The five files fit together like this:



3. Dataset File #1 — sales_train_validation.csv

What it contains: daily unit sales for all 30,490 store-item combinations, from day `d_1` (2011-01-29) through day `d_1913` (2016-04-24) — 1,913 days.

Why it exists: it appears to be an earlier snapshot of the same underlying sales table — everything in it (every ID, every value for the days it covers) is also present, unchanged, inside `sales_train_evaluation.csv`. It was verified that both files share an identical set of 30,490 base IDs in identical order.

What one row represents: one specific **product, in one specific store**, tracked over time. For example, the row with `id = HOBBIES_1_001_CA_1_validation` represents item `HOBBIES_1_001` sold in store `CA_1`, and nowhere else.

What the ID represents: the `id` column is built as `{item_id}_{store_id}_validation`, e.g. `HOBBIES_1_001_CA_1_validation`. It's a compound key — read left to right, it tells you the product and the store in one string.

What the `d_1`, `d_2`, `d_3`, ... columns represent: each one is a single calendar day, in sequence, starting from the very first day in the dataset. `d_1` is always 2011-01-29 for every row. The actual calendar date for `d_1913` (or any `d_n`) has to be looked up in `calendar.csv` — it is not stored in this file directly, only the sequence number is.

The wide-format structure, explained simply

```

item + store  (one row)
|
|— d_1 = units sold on day 1  (e.g. Saturday, 2011-01-29)
|— d_2 = units sold on day 2  (Sunday,    2011-01-30)
|— d_3 = units sold on day 3  (Monday,    2011-01-31)
|
|...
|— d_1913 = units sold on day 1913 (2016-04-24)

```

This is called **wide format**: instead of one row per (item, store, day) — which would be a ~58-million-row table — the file uses one row per (item, store), with **each day as its own column**. This makes the file compact and fast to scan, but any date-based operation (joining to the calendar, computing a rolling average, etc.) requires first "melting" the `d_` columns back into a normal (item, store, date, sales) long table, or working with the wide array directly using date-aligned column positions.

Item and store information is stored directly on the row, as separate columns — `item_id`, `dept_id`, `cat_id`, `store_id`, `state_id` — so no separate lookup table is ever needed to know which department or state a row belongs to.

WHY THIS MATTERS

Since every one of its rows and values is also contained in `sales_train_evaluation.csv` (next section), the *evaluation* file is the more complete — and therefore preferred — source of historical sales for building anything going forward.

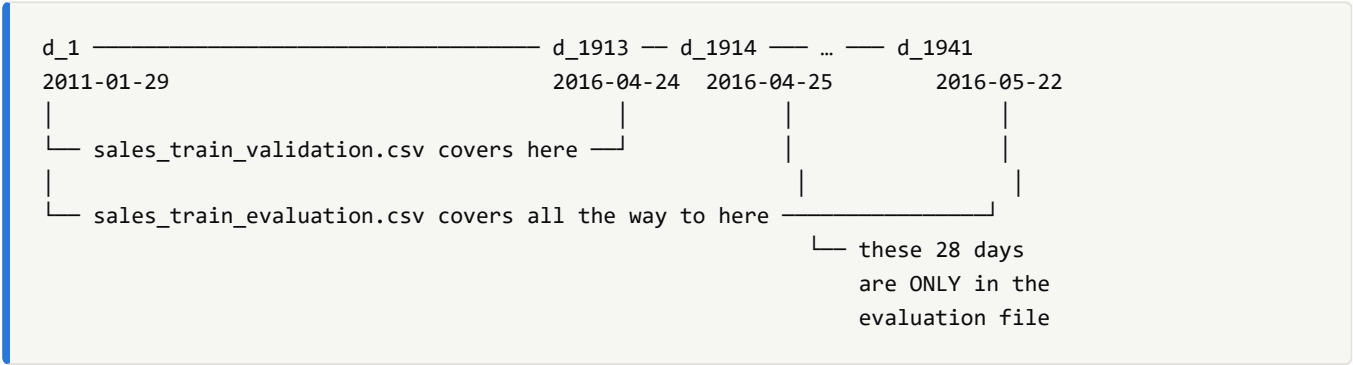
4. Dataset File #2 — sales_train_evaluation.csv

What it contains: the exact same 30,490 store-item structure as `sales_train_validation.csv` , but extended to day `d_1941` (2016-05-22) — **28 more days of real, actual sales** than the validation file has.

How it differs from sales_train_validation.csv

	sales_train_validation.csv	sales_train_evaluation.csv
Rows	30,490	30,490 (same IDs)
Day columns	<code>d_1</code> ... <code>d_1913</code>	<code>d_1</code> ... <code>d_1941</code>
Last date covered	2016-04-24	2016-05-22
id suffix	<code>_validation</code>	<code>_evaluation</code>
Relationship	Strict subset	Strict superset

Why the evaluation version exists — a timeline



This pattern is typical of a two-stage forecasting competition: an earlier round asked competitors to predict the 28 days from `d_1914` – `d_1941` using only the *validation* file as history. Those days' actual results have now been revealed and folded into `sales_train_evaluation.csv` , which is why the evaluation file looks like the validation file with 28 extra columns of real data.

ITS ROLE IN THIS PROBLEM

`sales_train_evaluation.csv` should be used as the **primary historical-sales source** — it has strictly more real ground truth, with no information lost. The genuinely unknown period we must predict is the **next** 28 days after evaluation's last day (`d_1942` – `d_1969` , i.e. 2016-05-23 → 2016-06-19) — not contained as real sales data in *any* file (see Section 10).

5. Dataset File #3 — calendar.csv

`calendar.csv` has **1,969 rows** (one per date) and **14 columns**. It turns the abstract `d_1`, `d_2`, ... day indices into real calendar information, including events and a food-assistance-benefit flag.

Column	Meaning	Example	Why it matters	Future dates?
<code>date</code>	Real calendar date	2011-01-29	Joins day-index to actual dates	Yes, all 1,969 rows
<code>d</code>	Day-index (sales file column names)	<code>d_1</code>	Join key to sales files	Yes, up to <code>d_1969</code>
<code>weekday</code>	Day name	Saturday	Weekend/weekday effect confirmed strong	Yes
<code>wday</code>	Numeric day-of-week, 1=Sat...7=Fri	1	Numeric version of weekday	Yes
<code>month</code>	Calendar month	1	Monthly/seasonal features	Yes
<code>year</code>	Calendar year	2011	Year-over-year features	Yes
<code>wm_yr_wk</code>	Internal year-week code	11101	Join key to <code>sell_prices.csv</code>	Yes, up to 11621
<code>event_name_1</code>	Special event name, if any	SuperBowl	Real demand driver on specific dates	Yes, verified non-missing for future 28 days
<code>event_type_1</code>	Category: Sporting/Cultural/National/Religious	Sporting	Groups events into 4 types	Yes
<code>event_name_2</code>	Second simultaneous event (rare, 5 dates total)	Easter	Handles rare double-event days	Yes
<code>event_type_2</code>	Category of <code>event_name_2</code>	Cultural	Same idea as <code>event_type_1</code>	Yes
<code>snap_CA</code>	SNAP usable in CA that day (0/1)	1	Government-benefit demand signal, esp. FOODS	Yes, verified non-missing
<code>snap_TX</code>	Same, Texas	0	—	Yes
<code>snap_WI</code>	Same, Wisconsin	0	—	Yes

WHAT IS "SNAP"? — INFERENCE, NOT A LABELED FACT

Based on the column names being state-specific, food-relevant, and following a recurring ~10-days-per-month pattern, `snap_CA` / `snap_TX` / `snap_WI` most plausibly represents days when SNAP (Supplemental Nutrition Assistance Program, a US food-assistance benefit) is disbursed/usable in that state. This is a **reasonable inference from context**, not something explicitly documented inside the file itself — treat the exact program definition as needing verification against the official problem statement if precision matters.

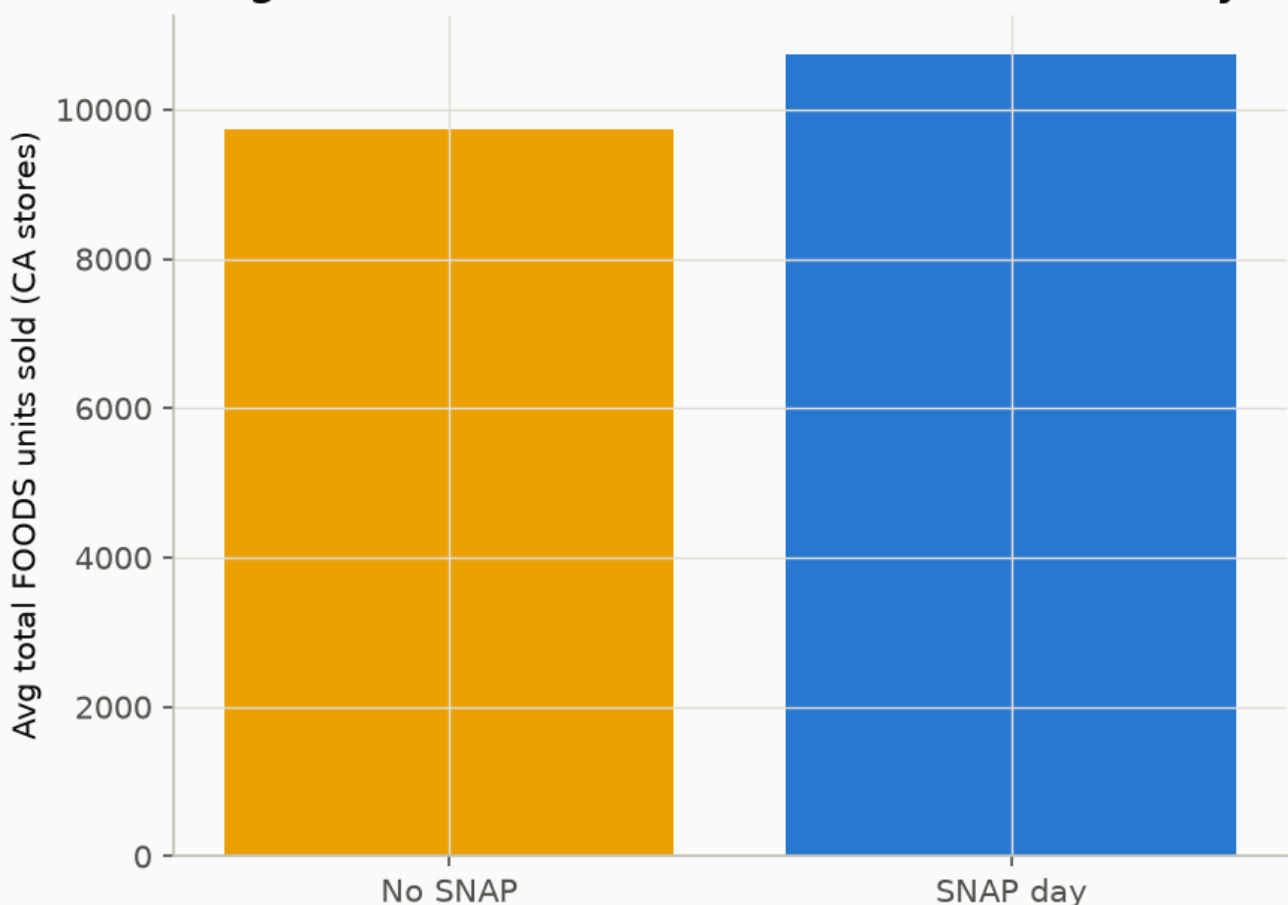
The most important thing about this file: it tells us about the FUTURE

```

Historical sales (d_1 ... d_1941)
      +
Future calendar information (d_1942 ... d_1969) ← ALREADY IN calendar.csv
      ↓
28-day forecast
  
```

Verified directly: `calendar.csv` has 1,969 rows (`d_1969` is its last), while the sales files stop at `d_1941` . The 28-day gap between them is precisely the forecast window, and every calendar/event/SNAP column in that window has **zero missing values**. This matters because the model isn't forecasting blind — it can "see" the calendar context of the days it's predicting for, even though it can't see the sales.

Avg CA FOODS units sold: SNAP vs non-SNAP days



Average CA FOODS units sold on SNAP vs. non-SNAP days — a single illustrative calendar-effect check from the investigation.

6. Dataset File #4 — sell_prices.csv

`sell_prices.csv` is the largest file by row count: **6,841,121 rows**, but only **4 columns**.

Column	Meaning	Example
<code>store_id</code>	Which store	CA_1
<code>item_id</code>	Which product	HOBBIES_1_001
<code>wm_yr_wk</code>	Which week (joins <code>calendar.wm_yr_wk</code>)	11325
<code>sell_price</code>	Price of that item, that store, that week	9.58

Why is price weekly instead of daily? This is exactly what the actual file structure shows: there is one price row per (`store_id` , `item_id` , `wm_yr_wk`) combination — there is no daily price column anywhere. The file itself was built at weekly granularity; a product's price is treated as constant for an entire week and only changes at week boundaries. This is a structural fact of the provided file, not an assumption.

Connecting price to daily sales

```

Store + Item + Week
    ↓
  Price  (looked up from sell_prices.csv via wm_yr_wk)
    ↓
Daily demand context
  (every day within that same week shares the same price,
   because calendar.csv maps each day to exactly one wm_yr_wk)
  
```

FUTURE PRICES ARE AVAILABLE — VERIFIED

`sell_prices.csv` 's `wm_yr_wk` values go up to **11621**, exactly matching `calendar.csv` 's maximum `wm_yr_wk` (the week containing the very last calendar day, `d_1969`). The last day of real sales (`d_1941`) falls in week 11617 — so price data extends 4 further weeks (28 days), **covering the entire forecast window**. We already know what price each product will be sold at during the 28 days we need to forecast.

THE PARTIAL PRICE-HISTORY ISSUE (FOUND DURING THE INVESTIGATION)

Not every store-item pair has a price for every one of the 282 distinct weeks in the file. The number of priced weeks per pair ranges from **19 up to the full 282** (only 35.9% — 10,932 of 30,490 pairs — have a price for literally every week). Cross-checked directly against sales: for item `FOODS_3_595` in store `CA_1` , whose price history starts at week `11603` , sales were **exactly zero for every day before that first priced week** (1,841 consecutive zero days), then 126 total units afterward. This strongly indicates the product wasn't being sold in that store yet — not that demand was zero. See Section 14.

7. Dataset File #5 — sample_submission.csv

Why it exists: it's a template showing exactly what shape a set of predictions must take. It contains no useful sales information itself — every forecast value in it is currently a placeholder `0`.

What its rows represent: one row per store-item combination, just like the sales files — but there are **60,980 rows**, exactly double the 30,490 store-item combinations, because the file has **two blocks**: one set of 30,490 rows with `id` ending in `_validation`, and another 30,490 ending in `_evaluation`. Both, once the suffix is removed, match the exact same 30,490 store-item universe as the sales files (verified).

What the F1–F28 columns mean: `F1` is "the forecast for day 1 of the horizon," `F2` is "day 2," and so on through `F28` — the 28-day-ahead forecast values, one column per day, in order.

Simple example of the required shape

id	F1	F2	F3	...	F28
HOBBIES_1_001_CA_1_validation	0	0	0	...	0
HOBBIES_1_001_CA_1_evaluation	0	0	0	...	0
HOBBIES_1_002_CA_1_validation	0	0	0	...	0
...					

HOW THIS CONNECTS TO MODELING LATER

Whatever forecasting approach is eventually used, its final output must be reshaped into exactly this structure — one row per store-item (times two, for the `_validation` / `_evaluation` blocks), 28 forecasted values per row, matching column names `F1 ... F28`. **No prediction is being built in this document** — this section only explains the target output format.

8. How the Five Files Work Together

A concrete, step-by-step walkthrough of how you'd answer "what do we know about item `HOBBIES_1_001` in store `CA_1` on a given day, and what do we know about the day we need to forecast?"

STEP 1 — FIND THE ITEM/STORE COMBINATION IN THE SALES DATA

Look up `id = HOBBIES_1_001_CA_1_evaluation` in `sales_train_evaluation.csv`. This row has 1,941 daily sales values (`d_1 ... d_1941`), plus its `item_id`, `dept_id`, `cat_id`, `store_id`, `state_id`.

STEP 2 — FIND THE CORRESPONDING DATE FOR ANY GIVEN DAY INDEX

Say we care about `d_500`. Look up `d = "d_500"` in `calendar.csv` to get the real `date`, `weekday`, `month`, `year`, and `wm_yr_wk` for that day.

STEP 3 — LOOK UP THE CALENDAR INFORMATION FOR THAT DATE

From that same `calendar.csv` row: was there an `event_name_1`? Was `snap_CA` active (since this row's store is in California)?

STEP 4 — LOOK UP THE CORRESPONDING WEEKLY PRICE

Take the `wm_yr_wk` found in Step 2, and find the row in `sell_prices.csv` where `store_id = CA_1`, `item_id = HOBBIES_1_001`, `wm_yr_wk` matches. That gives the price in effect on `d_500`.

STEP 5 — USE ALL AVAILABLE INFORMATION TO REASON ABOUT DEMAND

Now you have, for `d_500`: the actual sales value, the day of week, whether it was a holiday or SNAP day, and the price — a complete daily picture for that single item-store-day.

FOR THE FORECAST WINDOW (D_1942–D_1969)

The same Steps 2–4 all still work — because `calendar.csv` and `sell_prices.csv` both already contain rows for those days — but **Step 1 has no sales value to read**, because that's exactly what needs to be predicted.

9. Understanding the 30,490 Time Series

$$3,049 \text{ items} \times 10 \text{ stores} = 30,490 \text{ series}$$

This means the same product is tracked **separately in every store it's sold in**:

```
Store CA_1 + Item HOBBIES_1_001 → one time series (1,941 daily values)
Store CA_1 + Item HOBBIES_1_002 → a different time series
Store CA_2 + Item HOBBIES_1_001 → yet another time series (same item, different store)
Store TX_1 + Item FOODS_3_090 → another time series
...                               ... (30,490 total)
```

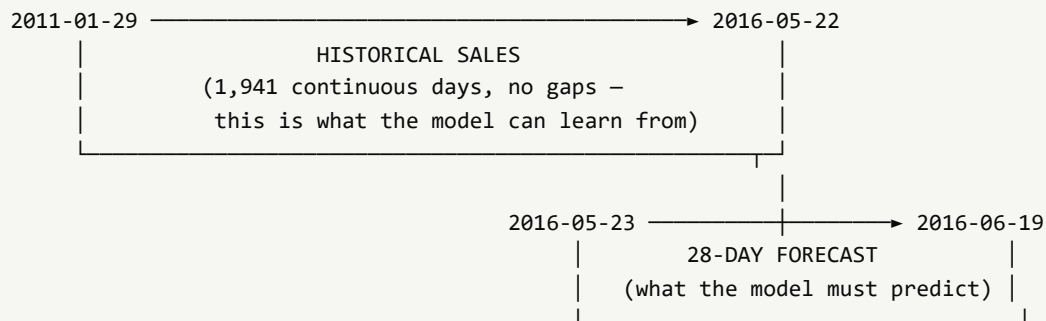
Each of these 30,490 combinations has **its own history, its own typical volume, and its own pattern of zero/nonzero days** — verified in the investigation to differ meaningfully by store (e.g., store **CA_3** alone generates 17% of total company-wide units sold, over 2.7× the lowest-volume store) and by item/category (FOODS items sell far more often than HOUSEHOLD or HOBBIES items).

WHY THIS MAKES THE PROBLEM MUCH HARDER THAN FORECASTING ONE PRODUCT

- A method that works well for a fast-moving item like **FOODS_3_090** in its best store (over 130 units/day on average) may not work at all for a slow-moving item that sells a total of 15 units across 5+ years.
- Any forecasting approach has to work — or be adapted to work — across this entire range of behavior, 30,490 times over, not just once.
- Aggregate patterns (like the overall upward trend or weekend effect in Section 12) may not hold uniformly for every individual series.

10. Date and Forecasting Timeline

All dates below come directly from `calendar.csv` and the day-column counts of the sales files.



Period	Dates	What it means
Historical	2011-01-29 → 2016-05-22	1,941 days of real, actual sales values, present in <code>sales_train_evaluation.csv</code>
Future / forecast	2016-05-23 → 2016-06-19	28 days. No sales values exist for this window in any file

What the model knows about the future window: the calendar (weekday, month, year, events, SNAP) and the price of every product in every store — both confirmed present for all 28 of these days (Sections 5 and 6).

What the model must predict: the actual unit-sales values for those same 28 days, for all 30,490 store-item series — precisely the `F1 ... F28` structure defined by `sample_submission.csv`.

11. The Future-Information Question

"If we only know historical sales, how can the model know about something happening in the future?"

With historical sales alone, the answer is: it can't. A model trained only on past `d_1 ... d_1941` sales values has no way of knowing, on its own, that (for example) June 19, 2016 is a particular day of the week, or that a SNAP benefit period is active that day, or what price the product will be sold at.

But this dataset gives us more than just historical sales:

```

Historical sales
+
Future calendar information (already in calendar.csv, for all 28 forecast days)
+
Future price information    (already in sell_prices.csv, for all 28 forecast days)
↓
A more informed forecast — the model can be given the exact weekday,
holiday/event status, SNAP status, and price for each of the 28 days
it needs to predict, even though it never sees the actual future sales.

```

Exactly what our dataset provides, stated plainly

Signal	Available for the future window?
Weekday, month, year	YES
Holiday/event names & types	YES
SNAP flags	YES
Price	YES
Actual future sales	NO (this is the target)

EXPLICITLY STATED: THERE IS NO PROMOTION DATASET IN THE PROVIDED FILES

A complete column-by-column scan of all five files found no promotion, discount, markdown, or "on-deal" flag anywhere. This means the model has no direct, labeled way to know "this product was on promotion that day" — a real driver of retail demand spikes that this dataset simply does not expose. Any such signal would have to be *indirectly inferred* from price changes (Section 14), which is a much weaker and noisier substitute than a true promotion flag.

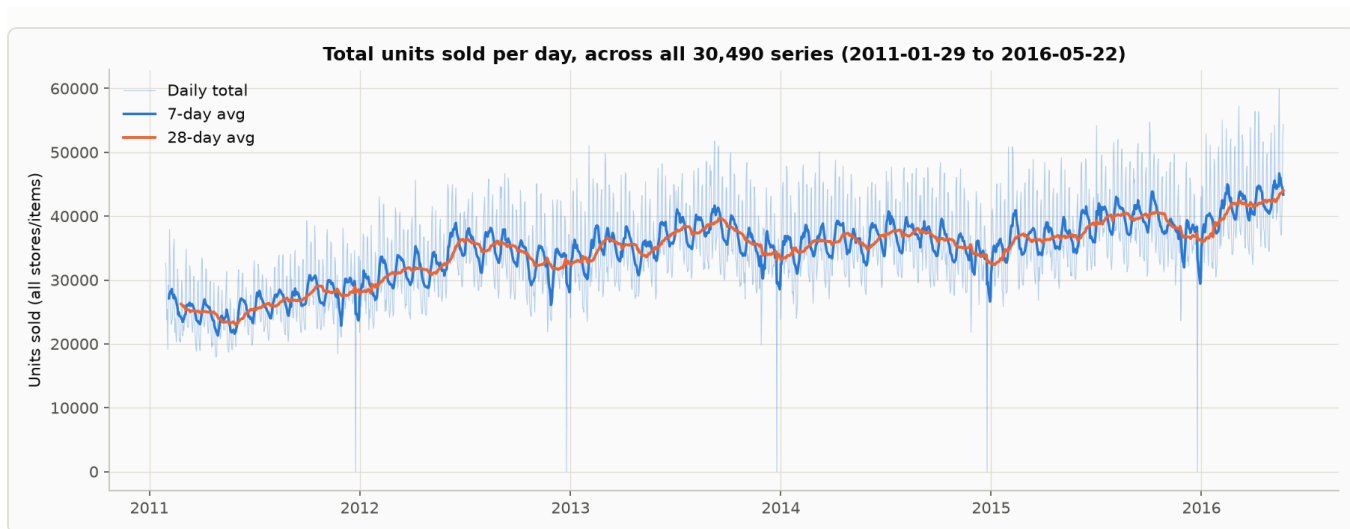
12. Data Characteristics We Discovered

Straight from the investigation, with a plain-language explanation of what each one actually means for us:

Finding	What it actually means for us
5.3 years of history (1,941 days)	Enough data to learn weekly patterns confidently, and enough calendar years (2011–2016) to attempt yearly-seasonality analysis later.
1,969 continuous calendar days, zero gaps	No missing/interpolated dates to worry about when building date-based features.
30,490 time series	A large panel of series, not one series — any approach needs to scale across all 30,490 combinations (Section 9).
~68% zero-sales observations	Most day-item-store combinations record no sale at all — the single most defining characteristic of the dataset (Section 13).
Strong intermittent-demand pattern	78% of all 30,490 series have <i>more</i> zero-days than nonzero-days across their whole history.
Partial price histories for ~64% of series	Roughly two-thirds of store-item pairs don't have a price for the entire timeline — lines up with the product being introduced later (Section 14).
No promotion data	Confirmed absent from all five files — a real limitation, not an oversight.
No negative sales values	The zero observations are genuine "nothing sold," not corrupted/clipped values — the data is trustworthy here.
No duplicate IDs	No accidental double-counting risk from duplicated rows.
No missing sales values	All 59,181,090 daily sales cells are non-null integers — no need to decide how to impute missing sales.

NOTE

This section is a summary of what was found — it deliberately does not recommend any particular modeling technique.



Total units sold per day across all 30,490 series, full history (2011-01-29 to 2016-05-22). The 7-day and 28-day rolling averages show a clear upward trend.

13. Zero Sales / Intermittent Demand

This is the most important characteristic of the dataset to understand before modeling, so it gets its own section.

A simple example — a week of sales for one product in one store

```
Day 1 → 0
Day 2 → 0
Day 3 → 0
Day 4 → 7
Day 5 → 0
Day 6 → 0
Day 7 → 12
```

Notice: most days show **zero** units sold, and then, seemingly out of nowhere, a day shows a real sale (7 units, then later 12 units). This pattern — long stretches of zero punctuated by occasional nonzero sales — is called **intermittent demand**.

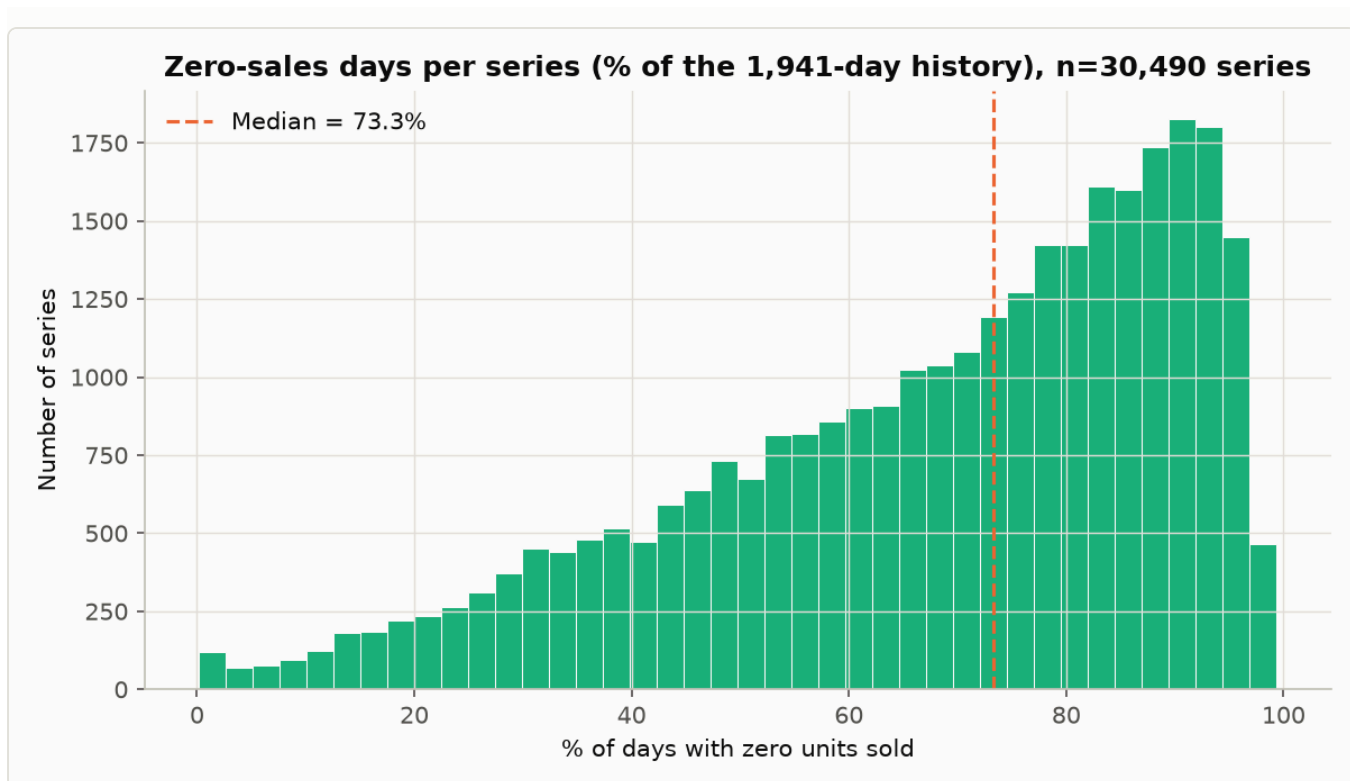
Why this is called intermittent demand: "intermittent" means "occurring at irregular intervals, not continuously." Unlike daily electricity usage (nonzero essentially every day, just varying in amount), many retail products are only purchased by a customer on *some* days — especially slower-moving items in smaller stores.

WHY THIS MATTERS FOR OUR DATASET SPECIFICALLY, USING THE REAL NUMBERS FOUND

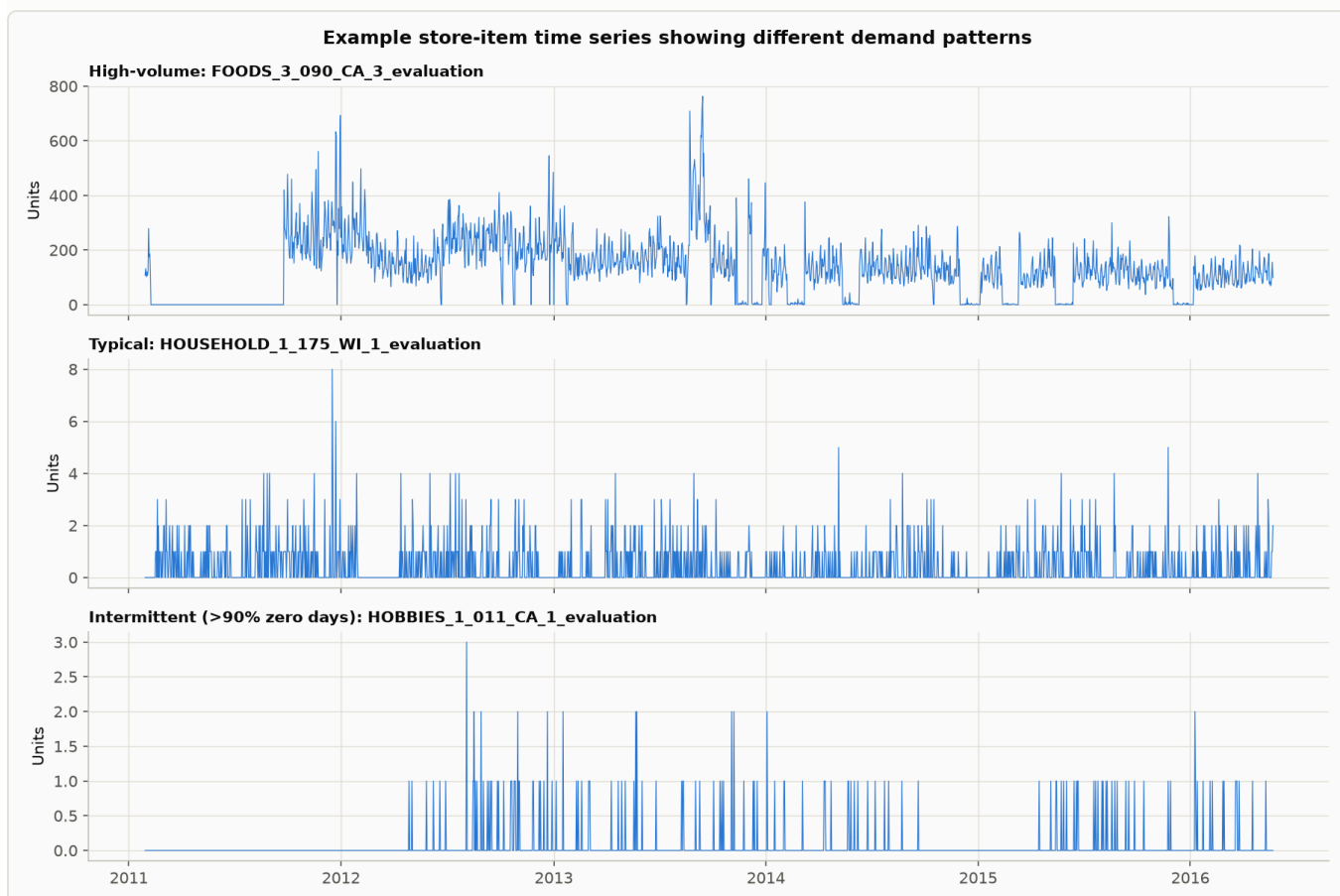
- **68% of all 59.18 million daily observations are exactly 0.**
- The median daily sales value, across the *entire* dataset, is **0** — more than half of all observations are zero.
- **Not a single one of the 30,490 series is "always selling"** — every store-item pair has at least one zero-sales day somewhere in its 1,941-day history.
- At the same time, **not a single series is "always zero"** either — every series has some nonzero sales at some point, so there's always a genuine signal to try to forecast, it's just sparse for many series.

WHAT THIS MEANS, WITHOUT PICKING A MODEL

The "typical" retail demand pattern here is closer to "occasional purchases" than "steady daily flow." A forecasting approach that assumes smooth, continuously-varying demand will be a poor fit for a large share of these series. This section deliberately does not recommend a specific modeling technique — understanding the pattern now is essential context for whatever comes next.



Distribution of the percentage of zero-sales days per series, across all 30,490 store-item series. Median is 73.3% zero-days.



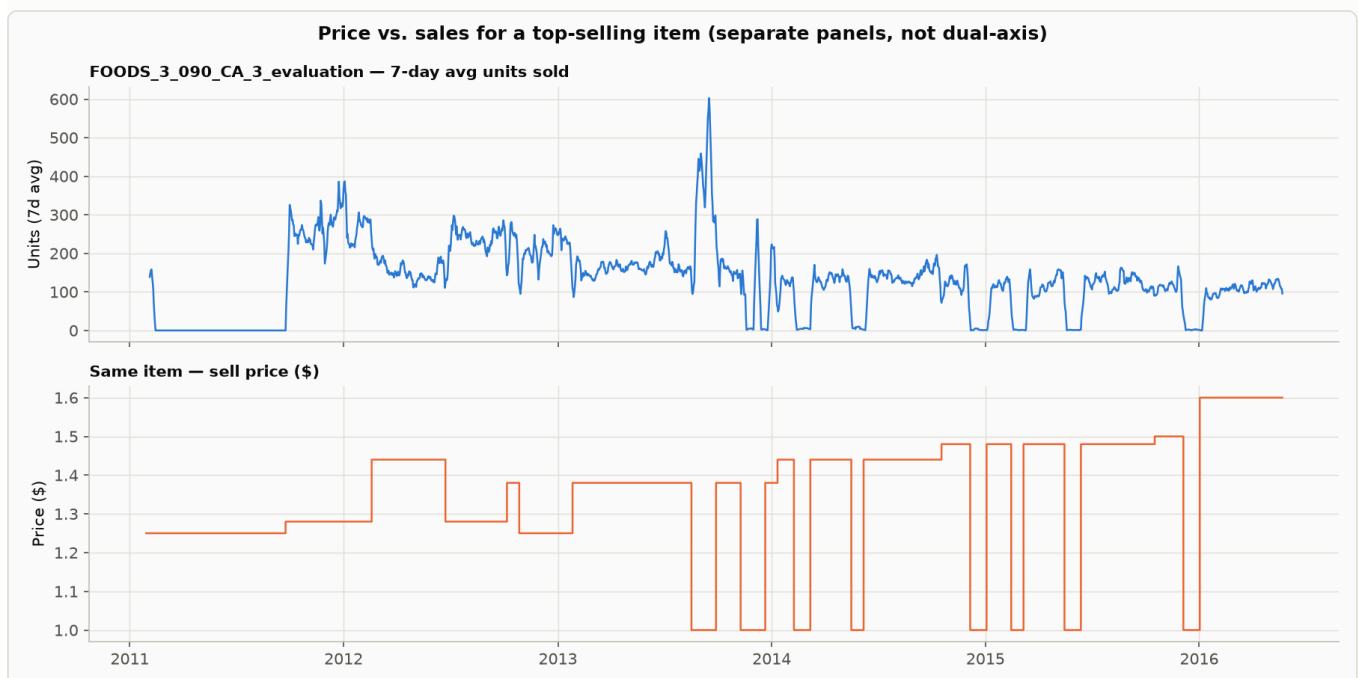
Three real example series from the actual dataset: a high-volume item, a typical item, and a heavily intermittent item (>90% zero-days).

14. Price Data Limitation

The investigation found that **19,558 of the 30,490 store-item pairs (64.1%) do not have a price recorded for every week in the dataset** — their price history starts later than the very first week.

WHY "MISSING PRICE HISTORY" DOES NOT AUTOMATICALLY MEAN "MISSING SALES"

It would be easy to assume a gap in price data is a data-quality problem. But when checked against actual sales for an example case — item **FOODS_3_595** in store **CA_1**, whose price record starts at week **11603** — the sales record showed **exactly 0 units sold for all 1,841 days before that first priced week**, and then real, nonzero sales (126 units total) after it. The zeros before the first price aren't missing data or errors — they're 100% consistent with the product simply not being available for sale in that store yet.



Price vs. sales for a top-selling item (FOODS_3_090_CA_3), shown as two separate time panels — not a dual-axis chart.

What this suggests: the investigation's working explanation — based directly on this cross-check, not a general assumption — is that **many products entered a given store's assortment partway through the 5.3-year timeline**, rather than being available from day one everywhere. A store-item pair with only 19 weeks of price history most likely represents a product introduced quite recently (relative to the full dataset), not a product with unreliable price records.

WHY THIS MATTERS GOING FORWARD

This "leading zero" block needs to be treated differently from an *ordinary* intermittent-demand zero (Section 13). A zero recorded *after* a product was already listed reflects genuine day-to-day demand variability. A zero recorded *before* a product's first price entry likely reflects "not for sale yet" — a different situation entirely that could bias a naive model if the two are not distinguished. This is flagged here as something to handle explicitly later — no fix has been applied to the data at this stage.

15. What Information Is Available at Forecast Time?

Information	Historically?	Forecast period?	Useful context	Notes
Historical sales	Yes (1,941 days)	No — the target	—	The exact thing being predicted
Calendar (date/weekday/month/year)	Yes	YES	High	Verified present for all 28 future days
Holidays / events	Yes	YES	Medium (sparse, 8.2% of days)	Non-missing for future window
Weekday	Yes	YES	High	Strong weekly pattern confirmed
SNAP flag	Yes	YES	Medium–High for FOODS	Verified non-missing, differs by state
Price	Yes	YES	High	Verified via matching wm_yr_wk coverage
Promotions	NO	NO	—	Not available in the provided dataset
Store	Yes (static)	Yes (static)	High	store_id, state_id
Product	Yes (static)	Yes (static)	High	item_id
Category / Department	Yes (static)	Yes (static)	Medium	cat_id, dept_id

FUTURE CALENDAR INFO

YES

FUTURE PRICE INFO

YES

PROMOTION DATA

NO

16. What We Are Not Given

Important Missing Information:

NO PROMOTION/DISCOUNT DATASET OR FLAG ANYWHERE

Confirmed by a full column-by-column scan of all five files. This limits the model's ability to explain sudden demand spikes commonly driven by promotions — any such effect can, at best, be weakly and indirectly approximated from price changes, not read from a genuine label.

NO INVENTORY OR STOCKOUT INFORMATION

There is no column indicating whether a product was actually in stock on a given day. A recorded 0 could mean "no one wanted to buy it" or "it wasn't available to buy" — the dataset cannot distinguish between these.

NO EXPLICIT "DATE ITEM WAS FIRST LISTED" COLUMN

The investigation *inferred* likely listing timing from the first available price week (Section 14), but there is no dedicated column that states this directly — it's a derived observation, not a labeled fact.

NO REVENUE/DOLLAR-SALES COLUMN

Only unit counts are given in the sales files; converting to revenue would require multiplying by the matching week's price — this has not been done anywhere in the provided files.

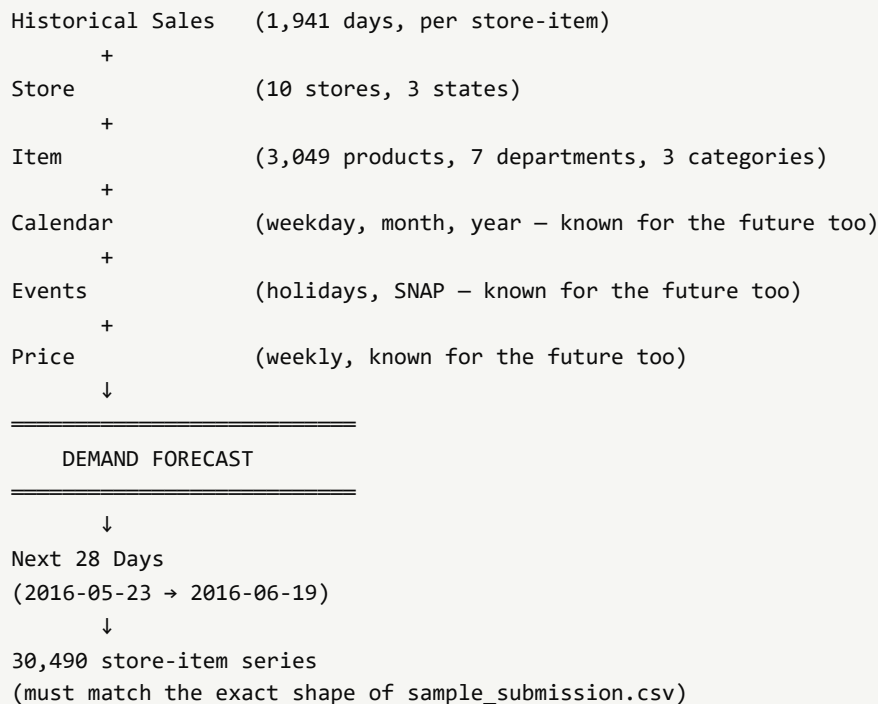
NO GROUND TRUTH FOR THE TRUE FORECAST WINDOW (D_1942–D_1969)

No file contains actual sales values for this period — expected and correct, since that's exactly what needs to be predicted, but worth stating plainly: there is nothing to "peek at."

Each of these gaps limits what any model — however sophisticated — can directly learn from labeled information; some of them (promotions, stockouts) may only ever be approximated indirectly, if at all.

17. Dataset → Forecasting Problem

Putting the whole picture together:



IN PLAIN ENGLISH

We're combining over five years of what actually happened (sales, broken down by exact product and exact store) with everything we already know is going to be true during the next 28 days (day of week, holiday, SNAP status, price) to estimate what hasn't happened yet — how many units of each of 30,490 product-store combinations will sell, one day at a time, for the next 28 days. **No modeling has been performed at this stage** — this section only shows how the pieces of the dataset connect to the forecasting task ahead.

18. Glossary

Store-item series

The sales history of one specific product in one specific store, tracked over time. There are 30,490 of these in this dataset.

Time series

A sequence of values recorded in order over time (here, one value per day).

Forecast horizon

How many time steps into the future you're predicting. Here: 28 days.

Demand

How much of a product customers wanted to buy; approximated in this dataset by recorded unit sales.

Intermittent demand

A demand pattern with many zero-value periods interrupted by occasional nonzero sales (Section 13).

Calendar feature

Any piece of information derived from the date itself (weekday, month, holiday, etc.) used to help a model understand time-based patterns.

Covariate

A variable other than the target itself that may help explain or predict it (e.g., price, weekday, SNAP status).

SNAP

In this dataset, a per-state daily flag plausibly tied to the Supplemental Nutrition Assistance Program benefit-usable days; the exact program definition is inferred from context, not explicitly documented in the files (see Section 5).

Event

A specific named occurrence on a calendar date (e.g. "SuperBowl," "LaborDay") that might influence shopping behavior, recorded in event_name_1/event_name_2.

Hierarchy

The nested grouping structure of products (item → department → category) and locations (store → state) present in this dataset.

Store

One of the 10 physical retail locations in the dataset.

Item

One of the 3,049 unique products in the dataset (also called item_id).

Department

A mid-level grouping of items (7 total, e.g. HOBBIES_1); several departments make up a category.

Category

The top-level product grouping (3 total: FOODS, HOBBIES, HOUSEHOLD).

Validation (file)

Here, `sales_train_validation.csv`, the earlier/shorter snapshot of the sales data (through 2016-04-24).

Evaluation (file/period)

Here, `sales_train_evaluation.csv`, the fuller snapshot of the sales data (through 2016-05-22); also used as a suffix in `sample_submission.csv` to denote the true, currently-unknown 28-day forecast block.

19. Final "Know This Before Modeling" Page

DATASET 5.3 years	PRODUCTS 3,049
STORES 10	STORE-ITEM SERIES 30,490
HISTORICAL SALES Through 2016-05-22	FORECAST WINDOW 2016-05-23 → 2016-06-19
HORIZON 28 days	ZERO SALES ~68%
FUTURE CALENDAR YES	FUTURE PRICES YES
PROMOTION DATA NO	MAIN CHARACTERISTIC Intermittent demand

Before choosing a model, we still need to determine:

1. How to treat `sales_train_validation.csv` vs. `sales_train_evaluation.csv` — and how to build a local validation split that doesn't leak the already-revealed `d_1914 – d_1941` window into training.
2. How to explicitly handle the "leading zero" blocks caused by products being introduced to a store partway through the timeline (Section 14), so they aren't confused with ordinary intermittent-demand zeros.
3. How strong the holiday/event and SNAP effects actually are, quantitatively — only a single illustrative check (SNAP vs. FOODS sales in California) has been done so far.
4. Whether price changes are a usable proxy signal for "something promotional happened," given there is no real promotion flag to rely on.
5. What approach is appropriate for a dataset this heavily zero-inflated — an open modeling question, intentionally not answered in this document.

This document does not build, train, or select a forecasting model. It exists solely to make sure the dataset itself is fully understood before that work begins.